

Module 15 – Agile Methodology

1. Introduction to Agile & Agile Manifesto

What is Agile?

Agile is an iterative and incremental software development methodology that emphasizes collaboration, customer feedback, flexibility, and continuous improvement.

Instead of delivering the entire project at the end, Agile divides work into **small, manageable iterations called sprints**, each producing a working increment of the product.

This allows teams to respond quickly to changing customer requirements and deliver value continuously.

Why Agile?

Traditional development approaches often struggle when requirements change during the project. Agile addresses this by promoting:

- Frequent software releases.
- Continuous customer involvement.
- Rapid feedback.
- Team collaboration.
- Adaptability to change.

Agile Principles

Agile focuses on:

- Delivering working software frequently.
- Encouraging collaboration between business stakeholders and developers.
- Responding to change rather than strictly following a fixed plan.
- Continuously improving products and processes.

Agile Manifesto

The **Agile Manifesto**, published in 2001, defines the core philosophy of Agile software development.

Four Agile Values

Agile Values	Explanation
Individuals and interactions over processes and tools	People and effective communication are more important than rigid processes.
Working software over comprehensive documentation	Delivering functional software is the primary measure of progress.

Customer collaboration over contract negotiation	Continuous customer involvement helps build the right product.
Responding to change over following a plan	Teams should adapt quickly when requirements evolve.

Note: The items on the right still have value, but Agile places greater emphasis on the items on the left.

Twelve Agile Principles (Summary)

The Agile Manifesto is supported by **12 guiding principles**, including:

1. Satisfy the customer through early and continuous delivery.
2. Welcome changing requirements, even late in development.
3. Deliver working software frequently.
4. Encourage daily collaboration between business and developers.
5. Build projects around motivated individuals.
6. Prefer face-to-face communication whenever possible.
7. Working software is the primary measure of progress.
8. Maintain a sustainable development pace.
9. Focus on technical excellence and good design.
10. Keep solutions as simple as possible.
11. Self-organizing teams produce the best results.
12. Regularly reflect and improve processes.

Agile vs Waterfall

Agile	Waterfall
Iterative and incremental development	Sequential development process
Requirements can change during development	Requirements are fixed before development begins
Frequent software releases	Single release at the end of the project
Continuous customer feedback	Limited customer involvement after requirements are finalized
Testing occurs throughout development	Testing is performed after implementation
Suitable for dynamic projects	Suitable for stable and well-defined projects

Advantages of Agile

- Faster delivery of usable software.
- Better customer satisfaction.
- Improved flexibility.
- Early identification of defects.
- Continuous improvement through regular feedback.
- Reduced project risk.

2. Scrum Framework

What is Scrum?

Scrum is one of the most widely used Agile frameworks for managing software development projects.

It organizes work into fixed-length iterations called **Sprints**, typically lasting **2 to 4 weeks**.

Each Sprint delivers a potentially shippable product increment.

Scrum Roles

1. Product Owner

The **Product Owner** represents the customer and is responsible for maximizing the value of the product.

Responsibilities include:

- Defining product requirements.
- Managing the Product Backlog.
- Prioritizing features.
- Communicating with stakeholders.
- Accepting completed work.

2. Scrum Master

The **Scrum Master** ensures that the Scrum process is followed correctly.

Responsibilities include:

- Facilitating Scrum events.
- Removing obstacles affecting the team.
- Coaching the team on Agile practices.
- Promoting continuous improvement.
- Protecting the team from external interruptions.

3. Development Team

The **Development Team** consists of professionals who build, test, and deliver the product.

Responsibilities include:

- Designing solutions.
- Writing code.
- Testing features.
- Fixing defects.
- Delivering working software each Sprint.

Scrum Ceremonies

Sprint Planning

Held at the beginning of each Sprint.

Objectives:

- Select backlog items for the Sprint.
- Define Sprint goals.
- Estimate tasks.
- Create the Sprint Backlog.

Daily Scrum (Daily Stand-up)

A short **15-minute daily meeting** where team members discuss:

- What was completed yesterday?
- What will be worked on today?
- Are there any obstacles?

The Daily Scrum helps keep the team aligned and identify blockers early.

Sprint Review

Conducted at the end of the Sprint.

Objectives:

- Demonstrate completed work to stakeholders.
- Gather customer feedback.
- Update the Product Backlog if necessary.

Sprint Retrospective

Held after the Sprint Review.

Objectives:

- Reflect on what went well.
- Identify areas for improvement.
- Agree on actions to improve the next Sprint.

Scrum Artifacts

Product Backlog

An ordered list of all desired product features, enhancements, bug fixes, and technical improvements.

The Product Owner continuously updates and prioritizes the backlog.

Sprint Backlog

A subset of Product Backlog items selected for the current Sprint, along with the tasks required to complete them.

The Development Team owns and updates the Sprint Backlog throughout the Sprint.

Increment

The **Increment** is the completed, tested, and potentially releasable software produced at the end of a Sprint.

Each Sprint adds new functionality to the previous increment.

Definition of Done (DoD)

The **Definition of Done** is a shared checklist that determines when a Product Backlog item is considered complete.

Typical criteria include:

- Code written.
- Code reviewed.
- Unit tests passed.
- Integration tests completed.
- Documentation updated.
- Product Owner approval received.

The DoD ensures consistency and quality across the team.

3. Agile Estimation & Planning Techniques

Story Points

Story Points are a relative measure of the effort required to complete a User Story.

They consider factors such as:

- Complexity.
- Amount of work.
- Risk and uncertainty.

Unlike hours or days, Story Points measure effort relative to other tasks.

Common Story Point values follow the Fibonacci sequence:

1, 2, 3, 5, 8, 13, 21, 34

Higher Story Points indicate greater effort or complexity.

Planning Poker

Planning Poker is a collaborative estimation technique used to assign Story Points.

Process

1. Product Owner explains the User Story.
2. Team members discuss requirements.
3. Each member privately selects a Story Point card.
4. All cards are revealed simultaneously.
5. Differences are discussed.
6. The team agrees on a final estimate.

This approach encourages participation and reduces estimation bias.

Sprint Planning Process

Sprint Planning typically involves:

1. Reviewing the Product Backlog.
2. Selecting high-priority User Stories.
3. Estimating Story Points.
4. Breaking User Stories into tasks.
5. Defining the Sprint Goal.
6. Creating the Sprint Backlog.

The outcome is a clear plan for the upcoming Sprint.

Velocity

Velocity measures the amount of work a team completes during a Sprint.

It is calculated as:

Velocity = Total Story Points Completed in a Sprint

Example:

Sprint	Story Points Completed
Sprint 1	28
Sprint 2	30
Sprint 3	32

Average Velocity:

$(28 + 30 + 32) / 3 = 30$ Story Points

Velocity helps teams estimate how much work they can realistically complete in future Sprints.

Burndown Chart

A **Burndown Chart** is a graphical representation of the remaining work in a Sprint over time.

It typically shows:

- **X-axis:** Sprint days.
- **Y-axis:** Remaining Story Points or tasks.

An ideal burndown line steadily decreases toward zero by the end of the Sprint.

Benefits include:

- Tracking Sprint progress.
- Identifying delays early.
- Monitoring whether the team is on schedule.

4. Agile User Stories

What is a User Story?

A **User Story** is a short, simple description of a software feature written from the perspective of the end user.

It focuses on **who** needs the feature, **what** they need, and **why** they need it.

User Story Format

The standard format is:

As a <user>,
I want <goal>,
So that <benefit>.

Example

As a customer,
I want to reset my password,
So that I can regain access to my account if I forget my password.

INVEST Principle

A well-written User Story follows the **INVEST** principle:

Letter	Meaning	Explanation
--------	---------	-------------

I	Independent	Can be developed without depending heavily on other stories.
N	Negotiable	Details can evolve through discussion.
V	Valuable	Delivers clear value to the user or business.
E	Estimable	The team can estimate the required effort.
S	Small	Can be completed within a Sprint.
T	Testable	Acceptance criteria can verify completion.

Acceptance Criteria

Acceptance Criteria define the specific conditions that must be satisfied for a User Story to be considered complete.

They remove ambiguity and provide a clear basis for testing.

Given–When–Then Format

Acceptance Criteria are commonly written using the **Given–When–Then** structure:

- **Given** – The initial context or precondition.
- **When** – The action performed by the user.
- **Then** – The expected outcome.

Example

User Story

As a registered user,
I want to log into the application,
So that I can access my account.

Acceptance Criteria

Given the user has a valid account,
When the user enters the correct username and password,
Then the system should successfully log the user in and display the dashboard.

Writing Effective User Stories

A good User Story should:

- Focus on user value.
- Be concise and easy to understand.
- Follow the INVEST principle.
- Include clear acceptance criteria.
- Be small enough to complete within a single Sprint.
- Encourage discussion between stakeholders and the development team.

Best Practices for Agile Teams

- Deliver working software in short, regular iterations.
- Prioritize high-value features in the Product Backlog.
- Hold Daily Scrums consistently to maintain team alignment.
- Keep User Stories small and independent.
- Estimate work collaboratively using Story Points and Planning Poker.
- Use the Definition of Done to ensure quality and consistency.
- Review completed work with stakeholders at the end of each Sprint.
- Conduct Sprint Retrospectives and implement agreed improvements.
- Continuously adapt to customer feedback and changing requirements.

Quick Revision Points

- **Agile** is an iterative and incremental software development methodology that emphasizes flexibility, collaboration, and continuous delivery.
- The **Agile Manifesto** is based on **4 core values** and **12 guiding principles**, promoting working software, customer collaboration, and responsiveness to change.
- **Scrum** is a popular Agile framework with three primary roles: **Product Owner**, **Scrum Master**, and **Development Team**.
- Scrum ceremonies include **Sprint Planning**, **Daily Scrum**, **Sprint Review**, and **Sprint Retrospective**.
- Scrum artifacts consist of the **Product Backlog**, **Sprint Backlog**, **Increment**, and the **Definition of Done (DoD)**.
- **Story Points** estimate effort relative to complexity, risk, and uncertainty, while **Planning Poker** is a collaborative technique for assigning those estimates.
- **Velocity** measures the amount of work completed in a Sprint, and a **Burndown Chart** tracks the remaining work over the Sprint duration.
- A **User Story** follows the format: *As a [user], I want [goal], so that [benefit]*.
- Effective User Stories follow the **INVEST** principle: **Independent, Negotiable, Valuable, Estimable, Small, and Testable**.
- **Acceptance Criteria**, often written using the **Given–When–Then** format, define the conditions that must be met for a User Story to be considered complete.